Mesh: A Token-Efficient, Graph-Native Logic for Repository-Scale Code Agents

Edgar Elmo Baumann · Philipp Nils Horn

Mesh · try-mesh.com

edgar.baumann@try-mesh.com · philipp.horn@try-mesh.com

June 2026

CONTENTS

References

1. Liu, N. F. et al. *Lost in the Middle: How Language Models Use Long Contexts.* TACL, 2024.
2. Ding, J. et al. *CrossCodeEval: A Diverse and Multilingual Benchmark for Cross-File Code Completion.* NeurIPS, 2023.
3. Liu, Y. et al. *RepoBench: Benchmarking Repository-Level Code Auto-Completion Systems.* ICLR, 2024.
4. Aider. *Building a Better Repo Map.* 2023.
5. Kamradt, G. *Needle in a Haystack — Pressure Testing LLMs.* 2023.
6. Ong, I. et al. *RouteLLM: Learning to Route LLMs with Preference Data.* arXiv:2406.18665, 2024.
7. Chen, L. et al. *FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance.* arXiv:2305.05176, 2023.
8. Robertson, S. & Zaragoza, H. *The Probabilistic Relevance Framework: BM25 and Beyond.* Foundations and Trends in IR, 2009.

9. Li, X. et al. *CoIR: A Comprehensive Benchmark for Code Information Retrieval.* arXiv:2407.02883, 2024.
10. Wang, L. et al. *CodeRAG-Bench: Can Retrieval Augment Code Generation?* Findings of NAACL, 2025.
11. Robertson, S., Zaragoza, H. & Taylor, M. *Simple BM25 Extension to Multiple Weighted Fields.* CIKM, 2004.
12. Anthropic. *Introducing Contextual Retrieval.* 2024.
13. Husain, H. et al. *CodeSearchNet Challenge: Evaluating the State of Semantic Code Search.* arXiv:1909.09436, 2019.
14. Guo, D. et al. *UniXcoder: Unified Cross-Modal Pre-training for Code Representation.* ACL, 2022.
15. Liu, S. et al. *CodeXEmbed: A Generalist Embedding Family for Code Search and Retrieval.* OpenReview, 2025.
16. Cormack, G. V., Clarke, C. L. A. & Büttcher, S. *Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods.* SIGIR, 2009.
17. Ferrante, J., Ottenstein, K. J. & Warren, J. D. *The Program Dependence Graph and Its Use in Optimization.* ACM TOPLAS, 1987.
18. Anonymous. *DKB: AST-Derived Graphs vs. LLM-Extracted Knowledge Graphs for Code RAG Reliability.* arXiv:2601.08773, 2026.
19. Edge, D. et al. *From Local to Global: A Graph RAG Approach to Query-Focused Summarization.* arXiv:2404.16130, 2024.
20. Luo, B. et al. *LightRAG: Simple and Fast Retrieval-Augmented Generation.* arXiv:2410.05779, 2024.
21. Gutiérrez, B. J. et al. *HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models.* NeurIPS, 2024.
22. Yao, S. et al. *ReAct: Synergizing Reasoning and Acting in Language Models.* ICLR, 2023.
23. Jimenez, C. E. et al. *SWE-bench: Can Language Models Resolve Real-World GitHub Issues?* ICLR, 2024.
24. Xia, C. S. et al. *Agentless: Demystifying LLM-Based Software Engineering Agents.* arXiv:2407.01489, 2024.
25. Microsoft Research. *GraphRAG: Unlocking LLM Discovery on Narrative Data.* 2024.
26. Cemri, M. et al. *Why Do Multi-Agent LLM Systems Fail? (MAST).* NeurIPS, 2025.
27. Khattab, O. & Zaharia, M. *ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT.* SIGIR, 2020.
28. Nogueira, R. & Cho, K. *Passage Re-ranking with BERT.* arXiv:1901.04085, 2019.
29. Lu, S. et al. *AdvTest: Adversarial Test Cases for Code Retrieval.* arXiv:2203.03850, 2022.
30. Voyage AI. *voyage-code-2: Code Retrieval Embeddings.* 2024.
31. Gao, L. et al. *Precise Zero-Shot Dense Retrieval without Relevance Labels (HyDE).* ACL, 2023.
32. Yan, S.-Q. et al. *Corrective Retrieval-Augmented Generation (CRAG).* arXiv:2401.15884, 2024.
33. Jiang, Z. et al. *Active Retrieval-Augmented Generation (FLARE).* arXiv:2305.06983, 2023.
34. Asai, A. et al. *Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection.* arXiv:2310.11511, 2024.
35. Jiang, H. et al. *LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models.* EMNLP, 2023.
36. Jiang, H. et al. *LongLLMLingua: Accelerating and Enhancing LLMs in Long-Context Scenarios via Prompt Compression.* arXiv:2310.06839, 2023.

37. Pan, Z. et al. *LLMLingua-2: Data Distillation for Efficient and Faithful Task-Agnostic Prompt Compression.* arXiv:2403.12968, 2024.
38. *HyperGraphRAG: Retrieval-Augmented Generation via Hypergraph-Structured Knowledge Representation.* arXiv:2503.21322, 2025.
39. *Cog-RAG: Cognitive Dual-Hypergraph Retrieval-Augmented Generation.* AAAI, 2025.
40. Wu, Q. et al. *AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation.* arXiv:2308.08155, 2023.
41. Hong, S. et al. *MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework.* arXiv:2308.00352, 2023.
42. Qian, C. et al. *ChatDev: Communicative Agents for Software Development.* arXiv:2307.07924, 2023.
43. Anthropic. *Prompt Caching.* 2024.
44. Google. *Gemini API Context Caching.* 2024.
45. Microsoft. *How GitHub Copilot Understands Your Workspace.* 2025.
46. Jimenez, C. E. et al. *SWE-bench Verified.* 2024.
47. *A Systematization of Knowledge on Prompt-Injection Attacks against Agentic Coding Assistants.* arXiv:2601.17548, 2026.
48. *Design Patterns for Securing LLM Agents.* arXiv:2506.08837, 2025.
49. Elastic. *Reciprocal Rank Fusion in Elasticsearch.* 2024.

Appendix A — Formal Notation Reference

The symbols below are used throughout the paper.

Symbol	Name	Definition
Q	Query	NL or identifier-anchored user/agent request at turn t
R	Repository state	snapshot of workspace files, mtimes, index version, embeddings, RIG edges
C	Candidates	ranked file/chunk set produced by retrieval fusion before rerank
B	Budget	remaining token (and optional USD) allowance for prompt construction
M	Model	selected LLM tier (e.g. Gemini Flash/Pro) invoked after pre-inference assembly
T	Tools	callable workspace actions (`semantic_search`, `read_file`, `rig_*`, …)
S	Session state	turn sequence, compressed history, tool-result ledger, artifact handles
P	Policy state	retrieval confidence, model route, cost accumulator, cache breakpoints
κ	Retrieval confidence	scalar in $[0,1]$ from `confidenceFromScore()`
$\Delta\kappa$	Top-margin	$\text{top}_1 - \text{top}_2$ score; a flat margin triggers recovery

Table A1. Core entities, session/policy state, and retrieval-confidence symbols.

Appendix B — Reproduction Checklist

Each result is reproducible from a script in `apps/cli/bench/`; the table maps each script to its output and the figure or table it produces.

Script (in apps/cli/bench/)	Produces	Output	Figure / Table
static-bench.mjs	compression tiers, 255 files (deterministic)	BENCH_RESULTS-STATIC.md	Fig. §12.1
scaling-bench.mjs	briefing vs corpus size (14–1,588 files)	BENCH_RESULTS-SCALING.md	Fig. §12.2
cc-vs-mesh/run-all-v2.mjs	3-arm localization head-to-head, n=229 (intent-routed)	results-all-v2-*.json	Fig. §12.3, Tables 3, 5, D1
niah-llm.mjs	needle-in-a-haystack file Hit@1 + cited-snippet tokens	niah-llm-*.json	Fig. §12.4
cc-vs-mesh/run-multiturn.mjs	multi-turn session economics (8 turns/repo)	results-multiturn.json	Fig. §12.5, Table 4
cc-vs-mesh/gen-tasks.mjs	auto-generate n=229 tasks with ground-truth gold	tasks-large.json	§11.1, §12.8
index-timing-sweep.mjs	index build time vs file count	index-timing-sweep-*.json	Fig. §12.7

Table B1. Reproduction map. The live E2E run requires generative-model credentials.

Appendix C — Glossary of Logical Terms

Cache prefix. The stable, byte-identical leading span of the assembled prompt — system rules plus the workspace briefing — held constant across turns so the provider can serve it from a prompt cache rather than re-encoding it (measured provider-prefix hit rate ≈65%).

Capsule. A schema-bound structural summary of one file produced by `buildCapsule()` — purpose, symbols, and test links packed under a fixed byte budget — used as the default briefing representation in place of full source.

Channel. One of the parallel evidence sources fused by `WorkspaceIndex.search()`: the field-weighted lexical (BM25F) channel, the dense chunk-vector channel, and the RIG graph-neighbor channel. On the n=31 suite the three channels tie on Hit@k; dense alone underperforms.

Chunk. The atomic unit of dense retrieval — a bounded span of a file whose surface text is enriched with signatures, routes, test names, and caller/callee context before embedding; its 768-dim vector persists in the `vectors.bin` sidecar keyed by chunk id.

Briefing. The workspace orientation block injected into the stable prompt prefix — an assembly of capsule summaries that holds at ~15.0K tokens, an absolute size independent of corpus size (≈52× below the 780K-token 255-file corpus; the ratio grows with the corpus).

Hit@k. A binary retrieval metric: 1 if the gold file's exact relative path appears among the top-k ranked paths, else 0 (stricter than basename matching, which collides on shared names like `index.ts`). Intent-routed Mesh reaches Hit@1 79.9%, Hit@3 91.7%, Hit@8 96.9% on the 229-task suite.

Locality unit. The true unit of evidence for a code agent — a chunk plus its k-hop neighbors in the dependency graph — rather than the whole repository; retrieving locality units instead of files is the structural premise of the evidence

layer.

Mode. A retrieval intent class (e.g. exact, debug, impact, conceptual) resolved by `routeRetrievalQuery()`; each mode shifts channel weights and static-signal boosts while keeping both lexical and dense channels active by default.

MRR. Mean reciprocal rank — the average of 1/rank of the first relevant (gold) result across tasks, rewarding higher placement than Hit@k captures; per-class Hit@k on the n=229 suite is reported in Table 3.

NIAH. Needle-in-a-haystack — a protocol that plants a target in a growing corpus to test retrieval depth vs context length. On a memorisation-proof variant (30 novel synthetic needles, haystacks to 1,488 files, n=120), Mesh's dense retrieval holds 95% file-level Hit@1 and stays flat as the haystack scales 10× (§12.4), where lexical search degrades from 26.7% to 10%.

RIG. The Repository Intelligence Graph — a deterministic, AST-derived graph of typed nodes (file, symbol, route, test) and edges (import, call, test-covers) that answers impact, dependency-tracing, and coverage queries chunks cannot, via bounded neighbor BFS.

Tier. A level in the deterministic compression ladder applied by the context layer — raw → compact → dense → semantic → emergency — trading fidelity for tokens (per-file 1.66× to 16.07×); the `semantic` AST-skeleton tier (3.05×) is by design less aggressive than the body-trimming `dense` tier (3.33×), so the ladder orders compact → semantic → dense → emergency (the escalation logic already applies this order).

TOON. Token-Oriented Object Notation — a tabular encoding for homogeneous tool-result arrays that is never larger than the JSON equivalent (26–58% fewer tokens in practice), making it opt-in safe rather than a new encoding risk.

κ (retrieval confidence). A scalar in [0,1] from `confidenceFromScore()` summarizing how strongly the top candidate is supported; it drives recovery and model-routing decisions.

Top-margin. The gap $\Delta\kappa$ between the first- and second-ranked scores (`retrievalMeta.topMargin`); a flat margin signals ambiguity and triggers iterative retrieval or an optional model-rerank recommendation.

Wrong-confident. The most dangerous failure mode — $\kappa \geq 0.55$ yet the gold file is not at rank 1, so the agent acts on a high-confidence wrong result without recovery; 3 such cases occur on the n=31 suite.

Under-retrieved. A recoverable failure where the gold file is absent from the top-8 results; 1 such case occurs on the n=31 suite (4 under dense-only).

Appendix D — Full Ablation & Per-Class Tables

The per-class view of the n=229 suite, consolidating the controlled retrieval quality (§12.3, §12.9) with the per-query token economics against the competent grep agent (§12.6) in one table. Mesh leads every class on Hit@1 and wins tokens on every class, the advantage widening from exact (−56%) to impact (−94%), where the graph tool answers in a compact dependency list rather than raw grep output.

Query class	n	Hit@1	Hit@3	Hit@8	Mesh tok	CC tok	Δ tokens
exact	120	82.5%	90.8%	97.5%	266	608	−56.3%
conceptual	56	69.6%	89.3%	94.6%	271	958	−71.7%
impact	53	84.9%	96.2%	98.1%	44	686	−93.6%
overall	**229**	**79.9%**	91.7%	96.9%	**216**	712	**−69.7%**

Table D1. Per-class retrieval quality and per-query token economics on the n=229 suite (intent-routed Mesh vs the competent grep agent; from `cc-vs-mesh`). Hit@k columns are the controlled-comparison configuration (§12.9); token columns are the §12.6 economics run [N1].

Appendix E — Environment-Flag Reference

The variables below strip individual layers for ablation. Each is read at search time, so a single run can isolate one layer without rebuilding the index.

Environment variable	Effect	Layer stripped
`MESH_LEXICAL_ONLY_SEARCH=1`	dense channel off (lexical only)	Evidence
`MESH_DENSE_ONLY_SEARCH=1`	lexical channel off (dense only)	Evidence
`MESH_DISABLE_GRAPH_EXPANSION=1`	RIG neighbor expansion off	Evidence
`MESH_ENABLE_EMBEDDINGS=0`	no vector-sidecar writes (lexical fallback)	Representation
`MESH_BENCH_RAW`	strip the entire Mesh scaffold (raw-agent baseline)	all

Table E1. Environment-flag reference (ablation toggles, verified against `workspace-index.ts`).

Appendix F — Notes

[N1] Two configurations. Retrieval-quality numbers (§12.3, §12.9) use the controlled-comparison configuration — full repository file set, NVIDIA `nv-embedcode-7B` dense embedding, symbol-definition matching for exact queries, the RIG graph for impact. Token-economics numbers (§12.5, §12.6) use the chunk-level production pipeline (NVIDIA `nv-embedcode-7b-v1`); the controlled per-class retrieval quality is reported in Table 3 and §12.9. The token advantage is an output-compactness effect (compact pointers vs raw grep/file content), independent of the embedding.

[N2] Lean output. The −69.7% per-query token figure uses Mesh's lean `semantic_search` output (path + purpose + one cited line, behind `MESH_LEAN_SEARCH_OUTPUT`); the verbose default payload (signals, citations, scores per hit) costs 731 tokens vs the grep agent's 712. The token win is an output-form property — Mesh returns compact pointers where the agent pulls raw file and grep content into context.

[N4] NIAH design. 30 novel needles — uniquely-named functions implementing a described behaviour the model cannot have memorised — planted one per file, queried by a paraphrase of the behaviour (never the symbol name), into haystacks of 150/400/800/1,488 real distractor files (n=120). Index-based retrieval ranks over the index rather than loading the haystack into a context window, so it sidesteps the lost-in-the-middle degradation that afflicts in-context long-context retrieval.

[N5] Symbol-survival measurement. Exported-symbol survival is measured on the 120 exact tasks: the answer-bearing symbol survives compression at 96–100% across all tiers, because the tiers always retain signature lines (the §7.2 invariant).

[N6] Same-model scaffold comparison (§12.10). LocBench_V1 subset, n=16 GitHub issues; each repository cloned at the buggy base commit, gold = the file(s) edited in the merged fix, hidden from both agents. Both arms run the *same* model — grep arm: ripgrep + file reads only; Mesh arm: the routed `semantic_search` retriever (`nv-embedcode-7B` dense over a code index, long issue queries routed to the conceptual/dense channel, the §12.9 configuration). **Opus 4.8:** grep Acc@1 15/16, Mesh 14/16, both Acc@5 16/16; McNemar p=1.0 (discordant 0:1, not significant); Mesh used fewer tool calls in 9/16 instances, grep in 4/16, 3 ties. **Haiku 4.5:** both arms Acc@1 15/16, Acc@5 16/16. Per-arm cost is counted from agent transcripts (tool-use blocks and output tokens). Raw retrieval recall on the same set, no agent loop: Hit@1 7/16 (43.8%), Hit@5 13/16, Hit@10 14/16 — lower than the §12.9 suite's 79.9% because LocBench queries are long, multi-symptom issue reports (the hardest input for any single-vector retriever) and the gold is frequently a deep helper module rather than the name-obvious file; the agent loop on top lifts this to the Table 7 accuracies. A parallel raw run on a 24-instance SWE-bench-Lite subset scored Hit@1 7/24, Hit@5 17/24 (raw recall on public repositories, reported as supplementary, not a resolve-rate claim). Reproducible from `apps/cli/bench/cc-vs-mesh/` .

Mesh · try-mesh.com · correspondence: edgar.baumann@try-mesh.com